embedding schemes (Su et al., 2021; Press et al., 2021) or novel activation functions (Shazeer, 2020). We thus performed a series of experiments to evaluate the benefit of each of these modifications for a causal decoder-only model in Le Scao et al. (2022). We adopted two architectural deviations in BLOOM:

ALiBi Positional Embeddings Instead of adding positional information to the embedding layer, ALiBi directly attenuates the attention scores based on how far away the keys and queries are (Press et al., 2021). Although ALiBi was initially motivated by its ability to extrapolate to longer sequences, we found it also led to smoother training and better downstream performance even at the original sequence length – outperforming both learned (Vaswani et al., 2017) and rotary (Su et al., 2021) embeddings.

Embedding LayerNorm In preliminary experiments training a 104B parameters model, we experimented with an additional layer normalization immediately after the embedding layer – as recommended by the `bitsandbytes`[17] library (Dettmers et al., 2022) with its `StableEmbedding` layer. We found this significantly improved training stability. Even though we also found it penalizes zero-shot generalization in Le Scao et al. (2022), we train BLOOM with an additional layer normalization after the first embedding layer to avoid training instabilities. Note the preliminary 104B experiments were conducted in `float16`, while the final training was in `bfloat16`. Since then, `float16` has been attributed as being responsible for many of the observed instabilities in training LLMs (Zhang et al., 2022; Zeng et al., 2022). It is possible that `bfloat16` alleviates the need for the embedding LayerNorm.

We represent the full architecture of BLOOM in figure 5 for reference.

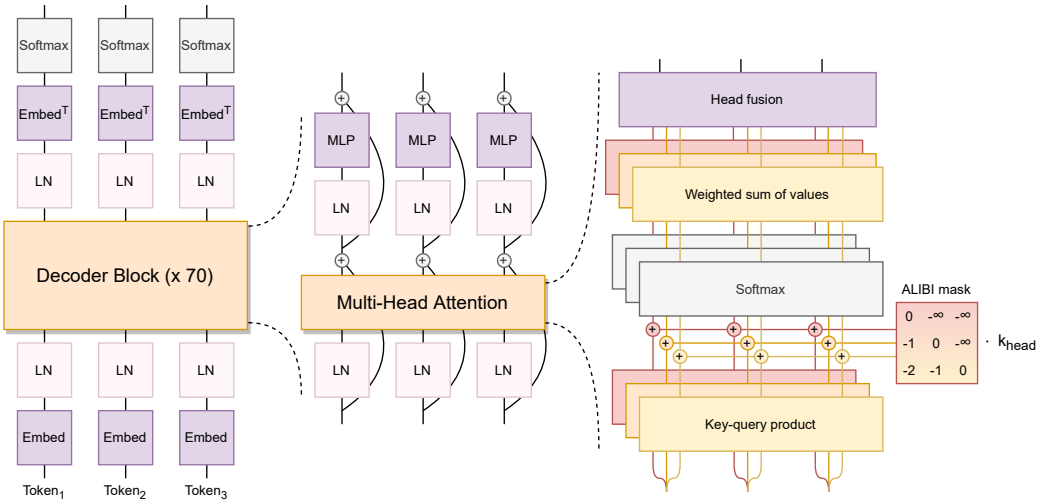

Figure 5: The BLOOM architecture. The k_{head} slope parameters for ALIBI are taken as $2^{\frac{-8i}{n}}$ with n the number of heads and $i \in 1, 2, ..., n$.

17. `github.com/TimDettmers/bitsandbytes`

3.3 Tokenization

The design decisions when training a tokenizer are often neglected in favour of "default" settings (Mielke et al., 2021). For instance, OPT (Zhang et al., 2022) and GPT-3 (Brown et al., 2020) both use GPT-2's tokenizer, trained for English. This can be justified by the fact that evaluating the impact of a particular choice on the downstream performance of the model is constrained by the large computational costs of training. However, the diverse nature of BLOOM's training data requires careful design choices to ensure that the tokenizer encodes sentences in a lossless manner.

Validation We use the fertility (Ács, 2019) of our tokenizer compared to existing monolingual tokenizers as a metric for sanity checks. Fertility is defined as the number of subwords created per word or per dataset by the tokenizer, which we measured using subsets of Universal Dependencies 2.9 (Nivre et al., 2017) and OSCAR (Ortiz Suárez et al., 2019) in the languages of interest. A very high fertility on a language compared to a monolingual tokenizer may indicate a degradation on the downstream multilingual performance of the model (Rust et al., 2021). Our goal was to not degrade the fertility on each language by more than 10 percentage points when comparing our multilingual tokenizer with monolingual tokenizers in corresponding languages. For all experiments, the Hugging Face Tokenizers library (Moi et al., 2019) was used to design and train the tested tokenizers.

Tokenizer	fr	en	es	zh	hi	ar
Monolingual	1.30	1.15	1.12	1.50	1.07	1.16
BLOOM	1.17 (-11%)	1.15 (+0%)	1.16 (+3%)	1.58 (+5%)	1.18 (+9%)	1.34 (+13%)

Table 2: Fertilities obtained on Universal Dependencies treebanks on languages with existing monolingual tokenizers. The monolingual tokenizers we used were the ones from CamemBERT (Martin et al., 2020), GPT-2 (Radford et al., 2019), `DeepESP/gpt2-spanish`, `bert-base-chinese`, `monsoon-nlp/hindi-bert` and Arabic BERT (Safaya et al., 2020), all available on the HuggingFace Hub.

Tokenizer Training Data We initially used a non-deduplicated subset of ROOTS. However, a qualitative study on the vocabulary of the tokenizer revealed issues in its training data. For instance, in earlier versions of the tokenizer, we found entire URLs stored as tokens caused by several documents containing a high number of duplicates. These issues motivated us to remove duplicated lines in the tokenizer training training data. We then applied the same sampling ratios per language as for the training data.

Vocabulary Size A large vocabulary size reduces the risk of over-segmenting some sentences, especially for low-resource languages. We conducted validation experiments using 150k and 250k vocabulary sizes to make comparisons with existing multilingual modeling literature easier (Conneau et al., 2020; Xue et al., 2021). We ultimately settled for a vocabulary of 250k tokens to reach our initial fertility objective compared to monolingual tokenizers. Since the vocabulary size determines the embedding matrix size, it also had to

be divisible by 128 for GPU efficiency reasons and by 4 to be able to use Tensor Parallelism. We used a final size of 250,680 vocabulary items with 200 tokens reserved for possible future applications such as removing private information using placeholder tokens.

Byte-level BPE The tokenizer is a learned subword tokenizer trained using the Byte Pair Encoding (BPE) algorithm introduced by Gage (1994). In order not to lose information during tokenization, the tokenizer creates merges starting from bytes as the smallest units instead of characters (Radford et al., 2019). This way, tokenization never results in unknown tokens because all 256 bytes can be contained in the vocabulary of the tokenizer. In addition, Byte-level BPE maximizes vocabulary sharing between languages (Wang et al., 2020).

Normalization Upstream of the BPE tokenization algorithm, no normalization of the text was performed in order to have the most general model possible. In all cases, we observed that adding unicode normalization such as NFKC did not reduce the fertility by more than 0.8% on all the languages considered but came at the cost of making the model less general; for example, causing 2^2 and 22 to be encoded in the same way.

Pre-tokenizer Our pre-tokenization has two goals: producing a first division of the text (usually using whitespaces and punctuation) and restricting the maximum length of sequences of tokens produced by the BPE algorithm. The pre-tokenization rule used was the following regex: "`ʔ[^(\s|[.,!?…。，、।۔،])]+`" [18] which splits words apart while preserving all the characters and in particular the sequences of spaces and line breaks that are crucial for programming languages. We do not use English-centric splits common in other tokenizers (e.g. splitting around `'nt` or `'ll`). We also didn't use splits on numbers and digits, which caused issues in Arabic and code.

3.4 Engineering

3.4.1 Hardware

The model was trained on Jean Zay,[19] a French government-funded supercomputer owned by GENCI and operated at IDRIS, the national computing center for the French National Center for Scientific Research (CNRS). Training BLOOM took about 3.5 months to complete and consumed 1,082,990 compute hours. Training was conducted on 48 nodes, each having 8 NVIDIA A100 80GB GPUs (a total of 384 GPUs); due to possible hardware failures during training, we also maintained a reserve of 4 spare nodes. The nodes were equipped with 2x AMD EPYC 7543 32-Core CPUs and 512 GB of RAM, while the storage was handled by mix of full flash and hard disk drives using a SpectrumScale (GPFS) parallel file system shared between all nodes and users of the supercomputer. 4 NVLink GPU-to-GPU interconnects per node enabled intra-node communications while 4 Omni-Path 100 Gbps links per node, arranged in an enhanced hypercube 8D global topology, were used for inter-node communications.

18. `github.com/bigscience-workshop/bs-tokenizers`
19. `idris.fr/eng/jean-zay/jean-zay-presentation-eng.html`